DESIGN REPORT

EMPLOYEE COMPENSATION MANAGEMENT SYSTEM

SESEIU24001 - Võ Châu Anh

---

1. Assumptions

These are the assumptions I used:

- Currency is VND, stored as `Double` and formatted by a `vnd` helper.
- `id` and `hireDate` are immutable (`let`) - employee identity never changes.
- `performanceRating` is a 0…5 score, set by HR, and used to compute the performance bonus.
- There are three employment arrangements: full-time, part-time, and contract. Contract has two sub-models: fixed monthly or milestone-based.
- Three user roles exist: `hrStaff`, `payrollAdmin`, `viewer`. Each role declares which actions it may perform via a permission check inside `Employee.authorize(_:_:action:)`.
- Payroll is monthly. `PayMonth` represents one calendar month and can be stepped via `.previous` / `.next`.
- Once a month's payment for an employee is approved, the payment record becomes locked. The only allowed change is a correction (`submitCorrection`) that appends a `Correction` with a written reason - the approved amount itself is never modified.
- No backend - `PayrollManager(sample: true)` loads six employees and last month approved for everyone, plus one sample correction; `PayrollManager.preview()` reuses it for SwiftUI previews.

2. Type identification

Classes:

Class	Responsibility
Employee	Base class. Holds shared identity (id, hireDate, name, department, position, performanceRating) plus the immutable payroll ledger (paymentRecords, correctionLog). Owns the payroll lifecycle: approvePayment, submitCorrection, finalPayable, and the authorize / ensureMonthOpen guards.
FullTimeEmployee	Fixed monthly salary + optional allowance + rating-based bonus.
PartTimeEmployee	Hourly rate × recorded work hours + rating-based bonus. Enforces hour limits.
ContractEmployee	Two contract models - fixed monthly payment or milestone-based. Manages milestone creation, amount editing, and approval.
PayrollManager	Owns all employees. Provides search, add, month summaries, approve-all, sample data.

Structs:

Struct	Fields	Responsibility
Milestone	let id = UUID(), fileprivate(set) var title: String, fileprivate(set) var amount: Double, fileprivate(set) var approvedMonth: PayMonth?, computed isApproved	One milestone under a contract. approvedMonth locks the milestone once paid.
WorkLog	let id = UUID(), let date: Date, let hours: Double	One hours-entry for a part-time employee.

Supporting types:

- ContractModel (enum) - fixed monthly / milestone-based
- EmploymentType (enum) - fullTime / partTime / contract
- UserRole (enum) - hrStaff / payrollAdmin / viewer, with .can(_:) permission check
- Permission (enum) - editInfo, editCompensation, approvePayment
- PayMonth (struct) - month identity with .previous / .next / .contains(date) / .date(day:)

- `PayLine` (struct) - one row on a payslip (label, amount)
- `PaymentRecord` (struct) - a frozen approved payslip
- `Correction` (struct) - post-approval adjustment (month, adjustment, reason, date)
- `PaySummary` (struct) - display model for the Payroll tab
- `InfoItem` (struct) - generic key-value row
- `PayrollError` (enum) - every validation error with a clear message

3. Properties & Methods

Shared stored properties in `Employee`:

- `let id`, `let hireDate` - immutable
- `private(set) var name`, `department`, `position`, `performanceRating`
- `private(set) var paymentRecords: [PaymentRecord]`
- `private(set) var correctionLog: [Correction]`

Overridable policy hooks in `Employee`:

- `employmentType`, `typeName`, `icon`, `bonusPolicyText`, `searchableText`, `compensationRows`
- `performanceBonusRate(for rating:) -> Double`
- `compensationLines(for month:) -> [PayLine]` - the main polymorphic method

Shared `final` methods (cannot be overridden by subclasses):

- `bonusLine(base:)`, `grossPay(for:)`, `displayLines(for:)`
- `record(for:)`, `isApproved(_:)`, `corrections(for:)`, `correctionsTotal(for:)`, `finalPayable(for:)`
- `authorize(_:_:action:)`, `ensureMonthOpen(_:)`
- `updateInfo(name:department:position:by:)`, `setPerformanceRating(_:by:)`
- `approvePayment(for:by:)`, `submitCorrection(for:adjustment:reason:by:)`, `overwriteApprovedPayment(for:newAmount:by:)` (always throws)

Subclass-specific stored properties (access level noted):

Class	Property	Access
`FullTimeEmployee`	`monthlySalary`, `allowance`	`private(set) var`
`PartTimeEmployee`	`hourlyRate`, `workLogs`	`private(set) var`
`ContractEmployee`	`contractModel`	`let`
`ContractEmployee`	`fixedMonthlyAmount`, `milestones`	`private(set) var`

Subclass-specific methods:

- `FullTimeEmployee.updateCompensation(salary:allowance:by:)`

- PartTimeEmployee.recordHours(_:on:by:), setHourlyRate(_:by:), hours(in:), logs(in:)
- ContractEmployee.setFixedAmount(_:by:), addMilestone(title:amount:by:), updateMilestoneAmount(id:newAmount:by:), approveMilestone(id:in:by:), approvedMilestones(in:)

searchableText is declared on Employee as an extension point but is not used by the current search (which uses id/name/department/position). This is a deliberate placeholder.

4. Inheritance & Polymorphism

Hierarchy:

```
Employee
├── FullTimeEmployee
├── PartTimeEmployee
└── ContractEmployee
```

All three subclasses are declared final, so no ad-hoc extension below them. Every shared behavior above is final, so subclasses can only override the policy hooks - this keeps payroll math consistent across types.

Inherited (written once): identity fields, paymentRecords ledger, correctionLog, the approve/correct workflow, authorize and ensureMonthOpen guards, bonusLine, grossPay, finalPayable, displayLines.

Complete override matrix:

Member	FullTimeEmployee	PartTimeEmployee	ContractEmployee
employmentType	.fullTime	.partTime	.contract
typeName	"Full-time"	"Part-time"	"Contract (Fixed payment)" / "(Milestone-based)"
icon	briefcase.fill	clock.fill	doc.text.fill
bonusPolicyText	rating-based text	rating-based text	model-dependent text
performanceBonusRate	20% if ≥ 4.5, 10% if ≥ 4.0, else 0	10% if ≥ 4.5, else 0	5% if milestone-based and ≥ 4.5, else 0

`compensationLines(for:)`	salary + allowance + bonus	hours × rate + bonus	fixed monthly amount or approved milestones + bonus
`compensationRows`	salary / allowance / policy	hourly rate / monthly limit / policy	contract model / fixed amount or milestone count / policy

`detailRows` in `Employee` is not overridden - it provides the shared identity rows (ID, type, department, position, hire date, rating). Subclasses contribute extra display info via `compensationRows` instead. This keeps the base class the single source of identity text.

Polymorphism in practice:

- `PayrollManager.summaries(for:)` calls `summary(of:month:)` for each `Employee`. Inside, `displayLines(for:)` resolves to the approved record or to `compensationLines(for:month:)`, which the actual type overrides. The manager never checks `is FullTimeEmployee` - each employee contributes its own lines.
- `finalPayable(for:)` sums the frozen record plus corrections. The base class does not care what type of employee produced the record.
- `EmployeeDetailView` does cast to specific types, but only to render the type-specific action forms (update salary, record hours, add milestone). The read-only payslip rendering is fully polymorphic.

When protocol/composition would be better: Swift cannot express "this employee is simultaneously full-time and contract-based". If the company later introduces hybrid arrangements, or an employee can hold multiple contracts at once, a class tree becomes awkward. In that case, protocols like `Payable`, `BonusEligible`, `MilestoneBased` and a `Contract` component would be cleaner. I chose class inheritance here because the three current types share one unified payroll lifecycle (approve, correct, lock) and the differences are entirely within `compensationLines` + `performanceBonusRate`.

5. Encapsulation & Validation

Access-control decisions:

Data	Access modifier	Reason
`Employee.id`, `hireDate`	`let`	Identity - must never change

`Employee.name`, `department`, `position`, `performanceRating`	`private(set) var`	HR must update them, but only via `updateInfo` / `setPerformanceRating` with validation and role check
`Employee.paymentRecords`, `correctionLog`	`private(set) var`	Ledger - changes only via `approvePayment` / `submitCorrection`
`FullTimeEmployee.monthlySalary`, `allowance`	`private(set) var`	Only `updateCompensation` may write
`PartTimeEmployee.hourlyRate`, `workLogs`	`private(set) var`	`hourlyRate` is written only by `setHourlyRate`, `workLogs` only by `recordHours`
`ContractEmployee.contractModel`	`let`	Fixed at creation
`ContractEmployee.fixedMonthlyAmount`, `milestones`	`private(set) var`	Only `setFixedAmount` / `addMilestone` / `updateMilestoneAmount` / `approveMilestone` may write
`Milestone.title`, `amount`, `approvedMonth`	`fileprivate(set) var`	Settable only inside `ContractEmployee.swift` - external code cannot change a milestone, and once `approvedMonth` is set the milestone is locked by the subclass
`PayrollManager.employees`	`@Published private(set)`	Views read the collection; only the manager appends
`PayrollManager.currentRole`, `selectedMonth`	`@Published` (writable)	The user picks these from the UI

Single gate for sensitive state: All mutating methods (`updateInfo`, `setPerformanceRating`, `approvePayment`, `submitCorrection`, `recordHours`, `updateCompensation`, `setHourlyRate`, `setFixedAmount`, `addMilestone`, `updateMilestoneAmount`, `approveMilestone`) are `final` in `Employee` or live only on a subclass. `overwriteApprovedPayment` is deliberately implemented to always throw - either `approvedPaymentLocked` (if approved) or `monthNotApproved`.

`ensureMonthOpen(_:)` is a shared guard called from three places: `approvePayment` (before creating a record), `PartTimeEmployee.recordHours` (before appending a work log), and `ContractEmployee.approveMilestone` (before setting `approvedMonth`). This stops hours and milestones from being changed in an approved month. Salary changes don't touch approved months because the PaymentRecord keeps its own copy of the pay lines.

Validation rules:

Rule	Where	Error
Name/department/position not empty (after trim)	`updateInfo`	`emptyField`
Rating in 0…5, finite	`setPerformanceRating`	`invalidRating`
Hours finite, ≥ 0, ≤ 24 per entry	`PartTimeEmployee.recordHours`	`invalidHours`
Monthly total ≤ 120	`PartTimeEmployee.recordHours`	`monthlyHoursExceeded`
Month must not be approved before adding hours	`recordHours` (via `ensureMonthOpen`)	`monthAlreadyApproved`
Salary > 0, allowance ≥ 0, both finite	`FullTimeEmployee.updateCompensation`	`invalidAmount`
Hourly rate > 0, finite	`PartTimeEmployee.setHourlyRate`	`invalidAmount`
Contract model must be fixed-monthly	`ContractEmployee.setFixedAmount`	`notFixedContract`
Contract model must be milestone-based	`addMilestone`, `approveMilestone`	`notMilestoneContract`
Milestone title not empty, amount > 0	`addMilestone`	`emptyField` / `invalidAmount`
Milestone must exist	`updateMilestoneAmount`, `approveMilestone`	`milestoneNotFound`

Approved milestone cannot be edited	`updateMilestoneAmount`	`milestoneLocked`
Approved milestone cannot be re-approved	`approveMilestone`	`milestoneAlreadyApproved`
Milestone cannot be approved in a locked month	`approveMilestone` (via `ensureMonthOpen`)	`monthAlreadyApproved`
No duplicate employee ID	`PayrollManager.addEmployee`	`duplicateEmployee`
Month must be open to approve	`approvePayment` (via `ensureMonthOpen`)	`monthAlreadyApproved`
Total must be > 0	`approvePayment`	`nothingToPay`
Correction requires a written reason	`submitCorrection`	`correctionReasonRequired`
Correction must be non-zero and keep final ≥ 0	`submitCorrection`	`invalidCorrection`
Correction requires the month to be approved	`submitCorrection`	`monthNotApproved`
Direct overwrite of approved payment	`overwriteApprovedPayment`	`approvedPaymentLocked`
Role must have the required permission	`authorize`	`unauthorized`

6. SwiftUI implementation

Screen structure:

- `ContentView` - `TabView` with Employees and Payroll. Injects `PayrollManager(sample: true)` as `@StateObject`.
- `EmployeesView` - search field, segmented filter by `EmploymentType`, `RoleBanner` (current role + selected month), and a list of `EmployeeCard`. Cards show type

color, name, department, current month's payable amount and Pending/Approved status.
- `EmployeeDetailView` - four stacked sections:
    - Information (shared `detailRows`) + Edit button.
    - Compensation details (polymorphic `compensationRows`) - automatically shows different rows for full-time, part-time, or contract.
    - Type-specific action form (`FullTimeSection` / `PartTimeSection` / `ContractSection`) - cast for the action form only; the payslip below is rendered polymorphically.
    - Monthly pay (`PaymentSection`) - month switcher, per-line amounts, corrections, net payable, an Approve button, an "Edit approved amount directly" button (always rejected), and a correction form.
- `PayrollView` - month switcher, total payable banner, "Approve all pending" button, and a list of `PayslipCard` for every employee.
- `EditEmployeeView` - sheet for updating name/department/position/rating. ID is shown as a locked `LabeledContent`, matching the `let` immutability in code.

Where polymorphic behavior is visible:
- Each employee card shows its own net payable for the selected month. In the payslip, full-time shows salary + allowance + bonus, part-time shows hours × rate, and contract shows the fixed amount or approved milestones.
- The Compensation details section renders different rows per type without any `switch` in the view - it just iterates `compensationRows`.
- Bonus policy text differs per type (`bonusPolicyText`), matching `performanceBonusRate`.
- Milestone-only controls (Approve, Add milestone) appear only on `ContractEmployee` detail - driven by the type cast for the action form, not by the payslip renderer.

7. Conclusion

The design keeps shared payroll behavior (identity, approve, correct, lock, net calculation) in `Employee` and pushes only pay-rule variance into subclasses via `compensationLines`, `performanceBonusRate`, and `compensationRows`. Sensitive data (`id`, `hireDate`, `paymentRecords`, `correctionLog`, salary fields, `milestones`) is guarded with `let` or `private(set)`/`fileprivate(set)`, and every mutating operation goes through a `final` gate that first calls `authorize(role, permission, action:)` and then validates input. Approved payments can only be adjusted through the correction procedure - direct overwrite is explicitly rejected by design. The result is a compact model that is easy to extend (a new arrangement is a new subclass with a few overrides) and easy to demonstrate in SwiftUI with clear valid/invalid feedback.